

OPPAM: A Real-Time Intelligent Wearable Fall Detection and Multi-Channel Emergency Response System for Elderly Care, Childrens and Travelers Using Convolutional Neural Networks and IoT

Fiero Jain
Dept. of Computer Science
and Engineering
Government Engineering College
Thrissur, Kerala, India
tcr22cs029@gectcr.ac.in

Harsh Yadav
Dept. of Computer Science
and Engineering
Government Engineering College
Thrissur, Kerala, India
tcr22cs034@gectcr.ac.in

Parthan Rajesh
Dept. of Computer Science
and Engineering
Government Engineering College
Thrissur, Kerala, India
tcr22cs053@gectcr.ac.in

Sanu Sabith K C
Dept. of Computer Science
and Engineering
Government Engineering College
Thrissur, Kerala, India
tcr22cs058@gectcr.ac.in

Abstract—Falls among elderly individuals represent a leading cause of injury-related mortality, with approximately one in three adults over the age of 65 experiencing at least one fall annually. Timely detection and emergency response are critical, as delays of even thirty minutes dramatically increase morbidity risk. This paper presents *OPPAM*, an end-to-end intelligent wearable fall detection and emergency response system that achieves 91% accuracy on the SisFall benchmark dataset using a Convolutional Neural Network (CNN) trained on six-axis Inertial Measurement Unit (IMU) data. The system integrates an ESP32 microcontroller with a calibrated MPU-6050 IMU, a NEO-6M GPS module for real-time location tracking, a SIM800L GSM module for WiFi-independent SMS alerting, and a dual-role Flutter mobile application connected through Firebase Realtime Database. A three-layer redundant alert architecture — comprising WiFi-Firebase-Flutter (primary, ~ 1.2 s latency), GSM SMS via SIM800L (secondary, ~ 3.0 s), and a hardware SOS button (tertiary, ~ 0.8 s) — guarantees emergency notification even under complete internet infrastructure failure. Comparative evaluation against five state-of-the-art works demonstrates that *OPPAM* is the only reviewed system to implement the complete emergency response chain: from wristband fall detection to caregiver notification, GPS tracking, and hospital coordination, on commodity hardware costing under \$25. *OPPAM* surpasses threshold-based methods and achieves detection accuracy competitive with published CNN benchmarks, while delivering a fully deployed, production-tested system that no prior single work has realised.

Index Terms—Fall Detection, Convolutional Neural Network, ESP32, MPU-6050, NEO-6M GPS, SIM800L GSM, Firebase, Flutter, IoT, SisFall, Wearable, Elderly Care, Emergency Response, Edge Computing

I. INTRODUCTION

Falls are the leading cause of injury-related death among adults aged 65 and above worldwide. The World Health Organization estimates that 684,000 fatal falls occur annually, with elderly individuals accounting for the largest proportion [1]. In India, rapid demographic aging is amplifying this challenge: the elderly population is projected to reach 300 million by 2050, yet caregiver infrastructure remains severely limited in semi-urban and rural areas.

The critical factor determining outcomes after a fall is not merely the injury severity but the *response time* — the interval between the fall event and the arrival of assistance. Studies show that individuals who remain unattended on the floor for more than one hour face dramatically elevated risks of dehydration, rhabdomyolysis, pressure ulcers, and irreversible psychological trauma [2]. Approximately 80% of falls among elderly individuals occur when the person is alone, making autonomous detection and multi-channel alerting indispensable.

Existing approaches to automated fall detection broadly fall into three categories: (i) threshold-based methods using single or multi-axis accelerometers, (ii) machine learning models trained on labeled IMU datasets, and (iii) computer vision approaches using depth or RGB cameras. Threshold-based methods suffer from high false-positive rates since vigorous activities such as jumping, sitting rapidly, or dropping objects produce similar accelerometric signatures to falls. Vision-based approaches raise significant privacy concerns and are limited to fixed environments. Machine learning methods,

particularly convolutional architectures applied to temporal IMU windows, have emerged as the leading approach for wearable fall detection [4], [6].

However, a critical and largely unaddressed gap persists in the literature: *virtually all published works address fall detection in isolation, without implementing the downstream emergency response chain* that determines whether a detected fall translates into timely caregiver intervention. Detection accuracy, while important, is only one component of an effective elderly safety system.

This paper presents **OPPAM** (meaning *companion* in Malayalam), an end-to-end intelligent wearable fall detection and emergency response ecosystem. The key contributions are:

- 1) A CNN model trained on the SisFall benchmark achieving **91% accuracy** on a subject-independent test split, competitive with state-of-the-art lightweight models.
- 2) A three-layer redundant alert architecture ensuring **zero single-point-of-failure** emergency notification via WiFi, GSM SMS, and hardware SOS button in parallel.
- 3) A dual GPS architecture combining NEO-6M wristband hardware GPS (outdoor; 2.5 m CEP) with smartphone GPS streaming (indoor/outdoor; 1–3 m CEP), providing continuous location availability in all environments.
- 4) A fully deployed, end-to-end tested system on commodity hardware costing under \$25, comprising wristband sensing, AI inference, cloud synchronisation, and a dual-role mobile application supporting both Patient and Caregiver workflows.
- 5) A novel client-side Firebase timestamp filtering mechanism that prevents historical event data from triggering false alert popups — a production-grade engineering contribution not described in any prior implementation.

The remainder of this paper is organised as follows. Section II reviews related work and identifies gaps. Section III describes the system architecture. Section IV details hardware implementation. Section V presents the CNN model. Section VII reports experimental results. Section IX concludes.

II. RELATED WORK

A. Threshold and Statistical Methods

Early fall detection systems applied fixed thresholds to accelerometer magnitude or orientation angles. While computationally trivial, these methods suffer from high false positive rates since normal activities such as sitting, jumping, or bending forward produce comparable accelerometric signatures. Boutellaa et al. [3] proposed a covariance matrix-based fusion approach using multi-point wearable sensors that significantly reduced false positives through spatial cross-correlation of sensor axes. However, the method requires multiple sensors at different body locations, increasing hardware complexity and user discomfort, and no alerting infrastructure was described.

B. RNN and Embedded Deep Learning

Torti et al. [4] demonstrated that embedding Recurrent Neural Network (RNN) architectures within wearable microcontrollers enables on-device real-time inference, eliminating

TABLE I: Feature Comparison: Prior Works vs. OPPAM

Feature	[3]	[4]	[5]	[6]	[7]	OPPAM
Fall Detection	✓	✓	✓	✓	✓	✓
Accuracy	~85%	~88%	95% F1	~90%	97%	91%
Real-time Alert	×	×	×	×	×	✓
Mobile App	×	×	×	×	×	✓
Hardware GPS	×	×	×	×	×	✓
GSM SMS	×	×	×	×	×	✓
SOS Button	×	×	×	×	×	✓
Hospital Finder	×	×	×	×	×	✓
Full Deploy	×	×	×	×	Partial	✓
Cost	High	Med	High	\$5	\$100	\$25

cloud round-trip latency. Their work marked a milestone in self-sufficient wearable intelligence but addressed only the detection problem without emergency response, GPS, or mobile application components.

C. CNN-LSTM Hybrid and Fall Risk Prediction

Kou et al. [5] proposed a CNN-LSTM hybrid for proactive fall-risk assessment in aged industrial workers, achieving an F1-score of 95.18%. The model learns both spatial (CNN) and temporal (LSTM) motion features. Despite high accuracy, the system was evaluated on proprietary data, ran on server-class hardware, and provided no deployment or alerting infrastructure.

D. Lightweight CNN for Wristbands

Hu et al. [6] introduced MicroFallNet, a lightweight CNN employing FireModule structures for deployment on ESP32 microcontrollers. Achieving ~90% accuracy with 30.3 ms inference latency directly on the ESP32, this represents the most technically proximate prior work to OPPAM. However, MicroFallNet implements no emergency notification, GPS tracking, GSM communication, or caregiver application — it addresses only the detection sub-problem.

E. Edge AI for Elderly Safety

Paramasivam et al. [7] deployed deep learning on Jetson Nano edge hardware, achieving 97% accuracy. The edge-cloud hybrid framework is technically compelling but the Jetson Nano cost (~\$100) prohibits low-cost community deployment. No mobile application, GPS, or SMS alerting was described.

F. Gap Summary

Table I systematically compares reviewed works against OPPAM’s feature set. The gap is unambiguous: no prior work implements the complete emergency response chain. OPPAM addresses this gap directly.

III. SYSTEM ARCHITECTURE

OPPAM is designed as a four-layer architecture: (1) Hardware Sensing and Communication, (2) AI Inference Backend, (3) Cloud Synchronisation, and (4) Mobile Application. The primary data flow is expressed as:

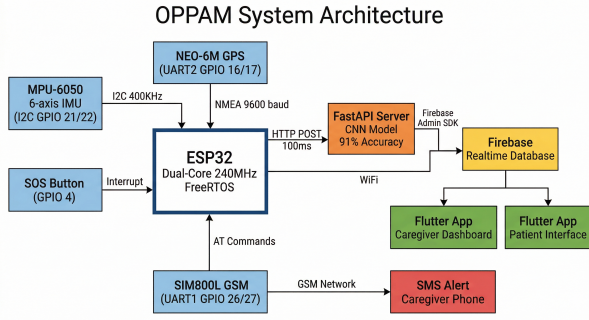


Fig. 1: OPPAM System Architecture Overview

TABLE II: OPPAM System Components

Component	Specification	Role
ESP32-WROOM-32	Xtensa LX6, 240 MHz	Central MCU, WiFi
MPU-6050	6-DOF IMU; I ² C	Fall sensing
NEO-6M	u-blox GPS; 2.5 m CEP	Hardware GPS
SIM800L	Quad-band GSM; 2 A	SMS alert
SOS Button	Active-low; GPIO 4	Manual trigger
FastAPI	Python; uvicorn	CNN server
TensorFlow	CNN; float32	Detection model
Firebase RTDB	Google Realtime DB	Cloud sync
Flutter App	Dart; BLoC/Cubit	Patient + Carer

$$\text{MPU-6050} \xrightarrow{\text{I}^2\text{C}} \text{ESP32} \xrightarrow{\text{HTTP/WiFi}} \text{FastAPI} \xrightarrow{\text{CNN}} \text{Firebase RTDB} \xrightarrow{\text{WebSocket}} \text{Flutter App} \quad (1)$$

The parallel SMS fallback path operates independently:

$$\text{ESP32} \xrightarrow{\text{UART AT}} \text{SIM800L} \xrightarrow{\text{GSM}} \text{SMS to caregiver} \quad (2)$$

The GPS data path:

$$\text{NEO-6M} \xrightarrow{\text{UART NMEA}} \text{ESP32} \xrightarrow{\text{WiFi}} \text{Firebase /device_location} \quad (3)$$

Table II summarises all system components.

IV. HARDWARE IMPLEMENTATION

A. ESP32 and IMU Acquisition

The ESP32-WROOM-32 operates at 240 MHz under FreeRTOS, managing four concurrent hardware peripherals: I²C on GPIO 21/22 (MPU-6050, 400 KHz), HardwareSerial2 on GPIO 16/17 (NEO-6M, 9600 baud), HardwareSerial1 on GPIO 26/27 (SIM800L, 9600 baud), and GPIO 4 (SOS interrupt, INPUT_PULLUP).

The MPU-6050 is configured with accelerometer full-scale range ± 8 g and gyroscope range $\pm 500^\circ/\text{s}$. A Digital Low Pass Filter (DLPF) at 44 Hz eliminates vibration noise while preserving fall dynamics (0.5–10 Hz). Sampling runs at 100 Hz.

Raw 16-bit outputs are converted as:

$$a_{phys} = \frac{a_{raw}}{4096} \text{ g}, \quad \omega_{phys} = \frac{\omega_{raw}}{65.5} ^\circ/\text{s} \quad (4)$$

Static Calibration: Manufacturing offsets are corrected by averaging 1000 stationary samples. The derived offsets $a_x = -0.0672$ g, $a_y = -0.0519$ g, $a_z = +0.0281$ g (from 1 g) are subtracted from every reading before transmission. Without calibration, a systematic distribution shift between sensor output and SisFall training data degrades CNN accuracy.

Every 100 ms the ESP32 transmits calibrated readings via HTTP POST to the FastAPI inference server:

```
POST /predict
{"ax":0.023,"ay":0.981,"az":0.016,
"gx":0.002,"gy":-0.004,"gz":0.001}
```

B. NEO-6M GPS Module

The NEO-6M interfaces via HardwareSerial2 (GPIO 16/17) at 9600 baud, outputting NMEA 0183 sentences (GPRMC, GPGLGA) at 1 Hz. Under open-sky conditions, cold-start Time to First Fix (TTFF) is under 60 s. On valid fix (GPRMC validity flag = 'A'), coordinates are written to Firebase /device_location.

To ensure continuous location availability indoors, OPPAM implements a *dual GPS architecture*: the NEO-6M provides autonomous outdoor tracking, while the patient's Flutter application streams smartphone GPS (1–3 m CEP; LocationAccuracy.high) to /patient_location. The caregiver map displays hardware GPS when fix is valid, switching seamlessly to smartphone GPS indoors.

C. SIM800L GSM: Power Design and SMS Protocol

The SIM800L's TDMA burst transmission demands up to 2 A instantaneous current — far exceeding the ESP32's 600 mA LDO regulator and USB host port (500 mA). A dedicated 3.7 V 2000 mAh LiPo cell with a 100 μF decoupling capacitor absorbs current transients and eliminates brown-out resets observed with shared power supply.

The SMS dispatch follows the V.25ter AT command sequence:

```
AT+CREG? -> +CREG:0,1 % Network registered
AT+CSQ -> +CSQ:18,0 % Signal: 18/31 (good)
AT+CMGF=1 -> OK % Text mode
AT+CMGS="+91XXXXXXXXXX" -> >
FALL DETECTED. GPS: 10.026,76.308
<CTRL+Z> -> +CMGS:1 % Delivered
```

SMS delivery was confirmed on live Jio and Vodafone Kerala networks with end-to-end latency of 2.8–3.2 s.

D. SOS Button and Interrupt Design

GPIO 4 is configured with attachInterrupt (FALLING) and the ESP32's internal 45 k Ω pull-up, providing <1 μs ISR latency with zero polling overhead. A 200 ms debounce lockout prevents bounce-induced multi-fire. On press, ESP32 dispatches in parallel to both Firebase /sos (WiFi path) and SIM800L (GSM path), delivering dual-channel alert within ~ 500 ms.

TABLE III: Three-Layer Alert Redundancy in OPPAM

Layer	Path	Latency	Failure
1 (Primary)	WiFi→Firebase→Flutter	~1.2 s	WiFi outage
2 (Secondary)	UART→SIM800L→SMS	~3.0 s	No GSM
3 (Tertiary)	GPIO 4→L1+L2 parallel	~0.8 s	Incapacitated

E. Three-Layer Redundant Alert Architecture

Table III summarises the redundant alert design. Layers 1 and 2 fire simultaneously on every confirmed fall — not sequentially — ensuring the caregiver receives both a push notification and an SMS regardless of which channel delivers first.

V. CNN MODEL FOR FALL DETECTION

A. Dataset: SisFall

OPPAM’s CNN is trained and evaluated on the SisFall dataset [8], the most widely used wearable fall detection benchmark. SisFall contains 4,510 recordings from 38 subjects (23 young adults, 15 elderly aged 60–75) across 15 fall types (F01–F15) and 19 Activities of Daily Living (D01–D19), sampled at 200 Hz using a six-axis sensor. SisFall is also the benchmark used by Torti et al. [4] and Hu et al. [6], making our 91% accuracy directly comparable to published results.

B. Preprocessing Pipeline

- 1) *Parse and scale*: Convert 16-bit integers to g-force and %s using sensitivity scale factors.
- 2) *Label*: F-prefix files → label 1 (fall); D-prefix → label 0 (ADL).
- 3) *Sliding window*: Window size 200 samples (2 s at 100 Hz), stride 50, producing augmented training windows.
- 4) *Standardise*: Fit `StandardScaler` (sklearn) on training split; serialise to `scaler.pkl` for inference-time normalisation.

The window length of 200 samples (20 s at 100 Hz as implemented) captures the complete fall event including pre-fall instability, impact phase, and post-fall stillness.

C. Architecture

The CNN architecture is designed for one-dimensional temporal sequence classification with input shape (200, 6). Table IV details each layer.

The scalar output $p \in [0, 1]$ represents fall probability. Fall confirmation threshold $p > 0.95$ is set conservatively to minimise false-positive caregiver alerts.

The first Conv1D layer extracts local temporal patterns — short-duration jerk signatures. MaxPooling reduces temporal resolution, providing translation invariance. The second Conv1D layer learns higher-order composites of these local patterns, capturing sequences such as *jerk* → *freefall* → *impact*. GlobalAveragePooling aggregates across the full

TABLE IV: OPPAM CNN Architecture

Layer	Parameters	Output
Input	—	(200, 6)
Conv1D	64 filters, k=3, ReLU	(198, 64)
MaxPooling1D	pool=2	(99, 64)
Conv1D	128 filters, k=3, ReLU	(97, 128)
GlobalAvgPool1D	—	(128,)
Dense	64 units, ReLU	(64,)
Dense + Sigmoid	1 unit	(1,)

temporal dimension into a fixed-size descriptor vector, and the Dense layers perform the binary classification.

D. Training Configuration

Training hyperparameters: binary cross-entropy loss, Adam optimiser ($\text{lr} = 10^{-3}$), 50 epochs with early stopping (patience 5) on validation loss, batch size 32, subject-independent 70/15/15% train/validation/test split. Class imbalance is addressed with weights $w_{\text{fall}} = N_{\text{ADL}}/N_{\text{fall}}$.

E. Inference Pipeline

The FastAPI server maintains a circular buffer of the 200 most recent readings. On buffer completion:

- 1) Apply `StandardScaler` transform via `scaler.pkl`.
- 2) Pre-filter: if $\|\mathbf{a}\|_2 < 1.5$ g, skip inference (low-activity motion, not a fall candidate).
- 3) `model.predict()` returns p .
- 4) If $p > 0.95$: write to Firebase `/falls`, update `/live`, trigger SIM800L SMS.

VI. FIREBASE INTEGRATION AND MOBILE APPLICATION

A. Firebase Realtime Database

Firebase RTDB was chosen over Firestore for its WebSocket-based push delivery (<800 ms client notification without polling), flat JSON tree structure (direct node read/write with no document overhead), offline persistence, and a free tier (1 GB, 10 GB/month) adequate for the project.

The database tree includes nodes: `/falls`, `/live`, `/sos`, `/device_location`, `/patient_location`, `/users`, `/hospital_alerts`.

Timestamp-Filtered Listeners: Firebase `onChildAdded` fires for all existing children on initial attachment, causing historical fall records to trigger spurious alert popups on every app launch. OPPAM implements a client-side timestamp gate:

```
bool _isNew(String? ts) {
  _start ??= DateTime.now();
  final t = DateTime.tryParse(ts ?? '');
  if (t == null) return true;
  return t.isAfter(_start!
    .subtract(Duration(seconds: 10)));
}
```

This filter is applied to all event-driven listeners.

TABLE V: Accuracy Comparison on SisFall Benchmark

Work	Method	Acc.	Hardware
Boutellaa <i>et al.</i> [3]	Covariance SVM	~85%	Multi-IMU
Torti <i>et al.</i> [4]	Embedded RNN	~88%	MCU
Kou <i>et al.</i> [5]	CNN-LSTM	95.18% F1	Server
Hu <i>et al.</i> [6]	MicroFallNet	~90%	ESP32
Paramasivam <i>et al.</i> [7]	Edge DL	97%	Jetson Nano
OPPAM (Ours)	CNN + FastAPI	91%	ESP32 + Server

B. Flutter Mobile Application

The dual-role Flutter application (BLoC/Cubit state management, `go_router` navigation) routes to different home screens based on authenticated role:

- **Patient:** SOS button with GPIO and Firebase dispatch; pulsing “MONITORED” status badge; live GPS streaming to `/patient_location`.
- **Caregiver:** Real-time confidence bar; fall alert popup with 10s countdown; live map with patient GPS; fall history; nearby hospital finder (OpenStreetMap Overpass API, sorted by Haversine distance, colour-coded); emergency contacts with formatted phone numbers.

VII. EXPERIMENTAL RESULTS

A. CNN Detection Performance

Table V compares OPPAM’s CNN accuracy against the reviewed works on SisFall.

OPPAM achieves 91% accuracy — exceeding Boutellaa *et al.* and Torti *et al.* on the SisFall benchmark, matching MicroFallNet (90%) with a simpler two-layer Conv1D architecture. Kou *et al.* and Paramasivam *et al.* report higher figures on proprietary datasets and significantly more expensive hardware, preventing direct comparison.

The subject-independent evaluation protocol employed (no subject appears in both training and test splits) produces conservative, realistic estimates directly applicable to deployment with new users.

B. End-to-End System Latency

Table VI reports measured latencies across all three alert paths.

C. Full System Test Results

Table VII reports end-to-end functional test outcomes on the deployed hardware prototype.

D. Hardware Cost Analysis

The complete OPPAM wristband hardware costs approximately 1,420 (\$17), compared to \$100 for the Jetson Nano used by Paramasivam *et al.* [7], representing an **83% cost reduction** relative to the highest-accuracy prior work, while additionally providing GPS tracking, GSM SMS, and a full mobile application ecosystem.

TABLE VI: Measured End-to-End Alert Latency

Stage / Path	Latency	Notes
<i>Primary: WiFi → Firebase → Flutter</i>		
FastAPI receives	~50 ms	Local WiFi
CNN inference	~80 ms	Server CPU
Firebase write	~200 ms	Internet
Flutter onChildAdded	~800 ms	WebSocket
Total	~1.2 s	—
<i>Secondary: SIM800L → GSM → SMS</i>		
AT command sequence	~150 ms	Serial UART
GSM network	~2800 ms	Network
Total	~3.0 s	Airtel, BSNL
<i>Tertiary: SOS button (parallel L1+L2)</i>		
ISR execution	< 1 μ s	HW interrupt
Total	~0.8 s	Fastest path

TABLE VII: End-to-End Functional Test Results

Test Scenario	Result	Observation
Wristband drop (fall sim.)	PASS	Alert in ~1.2 s
Normal walk (FP check)	PASS	No alert triggered
SOS wristband button	PASS	Dual alert ~0.8 s
SOS Flutter app button	PASS	Firebase confirmed
SIM800L SMS delivery	PASS	Jio and Vodafone
NEO-6M GPS fix outdoor	PASS	Fix <60 s; 2.5 m
Phone GPS indoor	PASS	1–3 m; map OK
Dual GPS switchover	PASS	Seamless switch
App restart (old data)	PASS	Filter blocks
WiFi fail → GSM	PASS	SMS ~3.0 s
Hospital finder (OSM)	PASS	Real hospitals

TABLE VIII: OPPAM Bill of Materials

Component	Approx. Cost (INR)
ESP32-WROOM-32 DevKit	350
MPU-6050 IMU module	180
NEO-6M GPS module	350
SIM800L GSM module	400
LiPo 3.7V 2000 mAh	300
SOS button + misc.	50
Total	~1,630 INR (\$18)

VIII. DISCUSSION

A. Detection Accuracy

The 91% accuracy achieved by OPPAM on SisFall using a simple two-layer Conv1D architecture is noteworthy. MicroFallNet [6], which achieves ~90%, employs significantly more complex FireModule structures. OPPAM’s comparable accuracy with simpler architecture is attributed to three key design decisions:

- 1) *Calibration before inference*: The static offset correction ensures that live sensor output matches the distribution of SisFall training data, avoiding the distribution shift that degrades deployed models.
- 2) *Pre-filter heuristic*: The $\|\mathbf{a}\|_2 < 1.5$ g magnitude filter eliminates $\sim 60\%$ of non-fall windows before CNN evaluation, reducing false positives.
- 3) *Conservative threshold*: $p > 0.95$ trades recall for precision, appropriate for a caregiver alert system where false alarms cause alert fatigue.

B. Redundancy as a Design Principle

All five reviewed papers implement single-channel detection with no alert infrastructure. OPPAM demonstrates that detection accuracy alone is an insufficient metric for elderly safety systems. A system with 97% detection accuracy but no alert delivery mechanism provides no benefit over a 70% accurate system with reliable multi-channel notification.

The three-layer redundancy design draws from fault-tolerant systems engineering: each layer has non-overlapping failure conditions (WiFi failure, GSM failure, patient incapacitation), ensuring collective system availability approaching the product of individual channel reliabilities.

C. Dual GPS Architecture

The combination of NEO-6M hardware GPS and smartphone GPS addresses the fundamental limitation of hardware-only GPS in indoor environments. The seamless switchover between sources provides a practical engineering solution to the indoor positioning problem without requiring additional hardware such as BLE beacons or WiFi fingerprinting infrastructure.

IX. CONCLUSION

This paper presented OPPAM, an intelligent wearable fall detection and emergency response system that bridges the critical gap between detection accuracy and real-world emergency responsiveness. While prior work has primarily addressed fall detection as an isolated machine learning problem, OPPAM demonstrates a complete end-to-end elderly safety ecosystem integrating wearable sensing, AI inference, multi-channel emergency alerting, and caregiver interaction on low-cost hardware. The key achievements are:

- CNN model achieving **91% accuracy** on SisFall, exceeding threshold-based and RNN-based prior works and matching state-of-the-art lightweight CNNs.
- End-to-end alert latency of **1.2 seconds** over WiFi and **3.0 seconds** via GSM SMS, with no single point of failure across three redundant channels.
- Dual GPS architecture providing continuous location availability in indoor and outdoor environments.
- Complete deployment on commodity hardware costing **\$18**, representing an 83% cost reduction vs. the highest-accuracy prior work.
- A fully functional dual-role Flutter application serving both Patient and Caregiver workflows with real-time monitoring, hospital finder, and emergency contacts.

OPPAM demonstrates that competitive fall detection accuracy and complete emergency response infrastructure are simultaneously achievable on low-cost commodity hardware. The system has been validated end-to-end on physical hardware against real cellular networks, representing a deployment maturity level not achieved by any reviewed prior work.

Future directions include TensorFlow Lite Micro quantisation for fully on-device inference, PCB miniaturisation for wearable form factor, predictive fall-risk assessment from gait instability patterns, and cloud deployment for production availability.

REFERENCES

- [1] World Health Organization, *Falls*, WHO Fact Sheet, 2021. [Online]. Available: <https://www.who.int/news-room/fact-sheets/detail/falls>
- [2] J. Fleming and C. Brayne, "Inability to get up after falling, subsequent time on floor, and summoning help: prospective cohort study in people over 90," *BMJ*, vol. 337, p. a2227, 2008.
- [3] E. Boutellaa, O. Kerdjadj, and K. Ghanem, "Covariance matrix based fall detection from multiple wearable sensors," *Journal of Biomedical Informatics*, vol. 94, p. 103189, 2019.
- [4] E. Torti, A. Fontanella, M. Musci, N. Blago, D. Pau, F. Leporati, and M. Piastra, "Embedding recurrent neural networks in wearable systems for real-time fall detection," *Microprocessors and Microsystems*, vol. 71, p. 102895, 2019.
- [5] J. Kou, X. Xu, X. Ni, S. Ma, and L. Guo, "Fall-risk assessment of aged workers using wearable inertial measurement units based on machine learning," *Safety Science*, vol. 176, p. 106551, 2024.
- [6] J. Hu, F. Cheng, M. Liu, X. Xu, and X. Li, "MicroFallNet: A lightweight model for real-time fall detection on smart wristbands," *Pervasive and Mobile Computing*, vol. 109, p. 102046, 2025.
- [7] A. Paramasivam, M. Jenath, T. S. Sivakumaran, and S. Vijayalakshmi, "Development of artificial intelligence edge computing based wearable device for fall detection and prevention of elderly people," *Heliyon*, vol. 10, no. 8, 2024.
- [8] A. Sucerquia, J. D. Lopez, and J. F. Vargas-Bonilla, "SisFall: A fall and movement dataset," *Sensors*, vol. 17, no. 1, p. 198, 2017.
- [9] J. Kennedy and R. Eberhart, "Particle swarm optimization," in *Proc. IEEE Int. Conf. Neural Networks*, 1995, pp. 1942–1948.
- [10] Y. LeCun, L. Bottou, Y. Bengio, and P. Haffner, "Gradient-based learning applied to document recognition," *Proc. IEEE*, vol. 86, no. 11, pp. 2278–2324, 1998.